How to use this document

This is a complete, self-contained build plan for **TrAItors**, a single-player social-deduction game for GitHub Game Off 2026 (theme: SHIFT). It is written so that any developer or AI coding assistant can implement it without further context. Work through the milestones in order; each has acceptance criteria that must pass before starting the next. When handing a milestone to an AI model, paste Sections 1–4 plus the single milestone you want built.

Hard constraints

- Deadline: **1 December 2026, 13:37 PST (21:37 UK)**. Code must be pushed to a public GitHub repo and the game submitted on itch.io separately before that time. Aim to submit by 29 Nov.
- Solo developer, ~29 days. Scope is deliberately small. Anything not in this document is out of scope unless every milestone up to M5 is complete.
- Stack is fixed: **TypeScript, Phaser 3, Vite**. No backend, no LLM calls, no network at runtime. The game must run as a static HTML5 build on itch.io.
- Deterministic simulation: all randomness goes through a seeded PRNG so a run can be replayed from its seed.
- Guiding rule for every decision: *cut anything that does not change the Round Table.*

1. Game summary

The player is the sole **Traitor** in a mansion with 6 AI **Faithfuls**. Each in-game day has three phases: **Day** (free roam the mansion under an action budget: plant evidence, chat, seed doubt, ally, bribe, while avoiding NPC sight cones), **Round Table** (NPCs discuss, accuse and vote one player out), and **Night** (the player murders one NPC in a room of their choosing). The player loses if banished; wins by surviving until two Faithfuls remain (final three).

Theme (SHIFT) is expressed four ways

- **Shift the blame** — the core verb (frame, plant, misdirect).
- **Shifting alliances** — a trust web between portraits that visibly redraws after every vote.
- **Night shift** — murders happen at night; alibis are about who was where and when.
- **The Shift key** — hold Shift to sneak in the mansion; hold Shift to lie in dialogue. Both have costs.

Win / lose

- Lose: receive the most votes at any Round Table (ties broken against the player).
- Win: reach the start of a day with exactly 2 Faithfuls alive, or survive 6 days, whichever comes first.
- Stretch (not planned): play as a Faithful.

2. Architecture

Separate the **simulation** (pure TypeScript, no Phaser imports, fully unit-testable) from the **presentation** (Phaser scenes for the mansion; plain DOM overlay for dialogue, the trust web and the vote). The sim exposes a small command API; Phaser and the DOM only call commands and render state.

```
src/
  main.ts                # boots Phaser, mounts DOM overlay
  sim/                   # PURE TS. No Phaser. Jest/Vitest tested.
    rng.ts               # seeded PRNG (mulberry32 or similar)
    types.ts             # all interfaces below
    world.ts             # rooms, adjacency, schedules
    npc.ts               # archetypes, memory, suspicion update rules
    evidence.ts          # items, provenance, planting, discovery
    sightings.ts         # record / recall with archetype noise
    rumours.ts           # seeded doubts propagate NPC->NPC
    roundTable.ts        # speech generation + vote resolution
    night.ts             # murder resolution, schedule reshuffle
    game.ts              # GameState + reducer: applyCommand(state, cmd)
    dialogue/
      templates.ts       # string templates with slots
      speak.ts           # picks template by archetype + suspicion band
  phaser/
    scenes/Boot.ts       # asset loading
    scenes/Mansion.ts    # tilemap, player, NPC sprites, sight cones, Shift-sneak
    scenes/Hud.ts        # day/phase/action-budget display
    entities/Player.ts
```

```
    entities/NpcSprite.ts
    systems/Vision.ts     # cone test -> sim.recordSighting
  ui/                     # DOM overlay (vanilla TS, no framework needed)
    dialogue.ts           # 4-verb menu, Shift-to-lie
    trustWeb.ts           # SVG web of suspicion lines
    roundTable.ts         # speeches, vote tally, banish animation
    night.ts              # pick victim + room
    endScreen.ts
  content/
    npcs.json             # 6 NPCs: name, archetype, portrait key, schedule
    rooms.json            # room ids, names, adjacency, tile bounds
    evidence.json         # 6 items with belongsTo
    templates.json        # dialogue lines
public/assets/            # tilemap (Tiled .json), tileset, sprites, portraits, sfx
tests/sim/                # vitest for everything in src/sim
```

Why this split: the suspicion engine is the game. Keeping it pure means it can be balanced with scripted tests ("plant item in room X, confirm Detective suspicion of Y rises") and tuned without touching rendering. It also means an AI model can implement *sim/* entirely from this document.

3. Data model (src/sim/types.ts)

```
type Id = string;
type Phase = 'day' | 'roundTable' | 'night' | 'ended';
type Archetype = 'detective' | 'sheep' | 'loyalist' | 'grudge' | 'paranoid';

interface Room { id: Id; name: string; adjacent: Id[]; }

interface Npc {
  id: Id; name: string; archetype: Archetype; alive: boolean;
  schedule: Record<number, Id>;          // dayTick -> roomId (ticks 0..5 per day)
  suspicion: Record<Id, number>;         // target (npc or 'player') -> 0..100
  memory: Sighting[];                    // what they think they saw
  lastAccusedBy?: Id;                    // for grudge archetype
  allyOf?: Id;                           // for loyalist archetype
  trusts: Record<Id, number>;            // -1..1, drives trust web + alliances
}

interface Sighting { observer: Id; subject: Id; room: Id; day: number; tick: number; confidence: number; }

interface Evidence {
  id: Id; name: string; belongsTo: Id;   // who it incriminates by default
  location: Id | 'player' | 'destroyed';
  plantedBy?: Id; plantedDay?: number;
  foundBy?: Id; foundDay?: number;
}

interface Rumour { source: Id; target: Id; claim: 'seenNear' | 'actedShifty' | 'hasMotive';
                   room?: Id; strength: number; day: number; isLie: boolean; }

interface Lie { day: number; toNpc: Id; claim: string; contradictedBy?: Id; }

interface GameState {
  seed: number; day: number; phase: Phase; tick: number;
  actionsLeft: number;                   // Day phase budget, default 6
  player: { room: Id; inventory: Id[]; lies: Lie[]; shield: boolean; };
  npcs: Record<Id, Npc>; rooms: Record<Id, Room>; evidence: Record<Id, Evidence>;
  rumours: Rumour[]; log: LogEntry[];    // human-readable event log for debugging/UI
  lastVote?: { votes: Record<Id, Id>; banished: Id };
  outcome?: 'win' | 'lose';
}

type Command =
  | { type: 'move'; to: Id; sneaking: boolean }
  | { type: 'pickUp'; evidence: Id }
  | { type: 'plant'; evidence: Id; room: Id }
  | { type: 'chat'; npc: Id }
  | { type: 'seedDoubt'; npc: Id; target: Id; room?: Id; isLie: boolean }
  | { type: 'ally'; npc: Id }
  | { type: 'bribe'; npc: Id }
  | { type: 'endDay' }
  | { type: 'resolveRoundTable' }
  | { type: 'murder'; victim: Id; room: Id };

function applyCommand(s: GameState, c: Command): GameState; // pure, returns new state
```

4. Simulation rules

4.1 Day phase

- Day has 6 ticks. Each NPC is in *schedule[tick]*. Moving between adjacent rooms costs 1 action and advances the tick by 1 for the player only when actions run out (ticks advance every 1 action; budget = 6 so the day ends when ticks do).
- **Sneaking** (Shift held): movement is 50% speed in Phaser; in sim, a sneaking move is only recorded as a sighting if the NPC's archetype-specific detection roll succeeds (Detective 70%, Paranoid 60%, others 30%). A non-sneaking move inside an NPC's cone is always a sighting.
- **Sightings** record (observer, subject, room, day, tick, confidence). Confidence starts at 1.0 and is degraded at recall time by archetype: Detective 0, Sheep 0.5 (may misreport room to a random adjacent one), Grudge 0.2 (may swap subject to whoever accused them last), Loyalist 0.1, Paranoid 0.1 (inflates confidence).
- **Evidence:** pickUp when in same room as item (1 action). plant places it in the current room (1 action). Each NPC entering a room has a 35% chance per tick to *discover* any evidence there (Detective 60%).
- **Discovery effect:** finder's suspicion of *belongsTo* += 25. **Provenance:** if the finder is the player (player reports it at Round Table), Detective/Paranoid add +15 suspicion of the player instead of only the target. If the finder has a sighting of the player in that room on that day, +20 suspicion of player.

4.2 Room verbs (1 action each, must be in same room as NPC)

Verb	Effect	Risk
chat	Reveals NPC's top-2 suspects and where they were last tick (filtered by their memory noise).	None.
seedDoubt	Creates a Rumour(source=player, target, strength 15). NPC suspicion of target += strength × trust[player]. If isLie, add to player.lies.	If a later sighting/evidence contradicts a lie, the Detective sets suspicion of player += 30 and tells the table.
ally	trusts[player] += 0.4. Loyalist: sets allyOf=player (votes with player). Paranoid: suspicion of player += 10.	Paranoid backfire.
bribe	Consumes a Mission reward token (if any). Target will not vote for player this day.	Sheep may mention it at the table (30%).

4.3 Rumour propagation (end of each tick)

- Any two NPCs in the same room exchange their strongest rumour with probability 0.5. The receiver adds a copy with strength × 0.6 and applies suspicion += strength × trust[source].
- Rumours decay by 3 strength per day and are removed below 3.

4.4 Round Table

- Each living NPC produces 1–2 speech lines from templates: an accusation of their top suspect, an alibi claim, a defence of their ally (Loyalist), or a complaint about who accused them (Grudge).
- **Player turn:** choose one statement from a menu (accuse X / claim alibi in room R / defend X / stay quiet). Holding Shift lets you pick a statement that is false; it is logged as a Lie. Truthful statements the Detective can verify give suspicion of player –5.
- **Vote resolution**, per NPC: Loyalist votes with allyOf's vote if ally alive, else top suspect. Sheep votes for whoever currently has the most votes (resolved after others; ties → own top suspect). Grudge votes for lastAccusedBy if set. Paranoid votes for whoever received the most *defences*. Detective votes for own top suspect. Player votes last and freely.
- Most votes is banished. Ties broken: player loses ties; otherwise random by seed. If banished is the player → outcome=lose.
- After the vote: trusts update (+0.2 between NPCs who voted the same way, –0.3 toward whoever voted for you). Trust web redraws from *trusts*.

4.5 Night phase

- Player chooses a victim and a room from the victim's schedule. Victim.alive=false.
- Body room is announced at the next Day start. Any NPC with a sighting of the player in that room on the previous day: suspicion of player += 30. Any NPC with a sighting of another NPC there: += 15 of that NPC.

- Dead NPC's schedule slots are redistributed; each survivor's suspicion of all is normalised so the max is 80 (prevents runaway locks).
- Win check at Day start: alive Faithfuls == 2 or day == 7 → outcome=win.

4.6 Tuning knobs (keep in one *balance.ts* file)

Detection rolls, discovery chance, suspicion deltas (+25 discovery, +30 contradiction, +30 body-room sighting), rumour strength and decay, normalisation cap, action budget, days to win. All numbers above are starting values; expect to tune in M5.

5. Content spec

5.1 Rooms (one floor, 9 rooms)

```
hall (hub) - adjacent: lounge, dining, library, stairsLanding
lounge - hall, conservatory        dining - hall, kitchen
kitchen - dining, cellar           cellar - kitchen
library - hall, study              study - library
conservatory - lounge              stairsLanding - hall, (bedroomCorridor as a single room)
bedroomCorridor - stairsLanding    (bedrooms are doors off the corridor, not separate rooms)
```

5.2 NPCs (6)

Name	Archetype	Personality hook (for dialogue templates)
Marcus	detective	Retired inspector. Precise, quotes times. Never forgets a sighting.
Priya	sheep	Anxious. Agrees with the room. Misremembers rooms.
Dalton	loyalist	Bonded to whoever allies first. Defends them at the table.
Imogen	grudge	Takes accusations personally. Votes for her last accuser.
Theo	paranoid	Distrusts popularity. Suspects whoever is defended most.
Rosa	detective	Second detective, lower detection roll (50%) for balance.

5.3 Evidence (6)

Torn letter (belongsTo Dalton), Cellar key (Theo), Bloodied cloth (none — incriminates whoever is seen with it), Photograph (Imogen), Poison vial (Rosa), Gloves (Priya). Each starts in a room from rooms.json.

5.4 Dialogue templates

Plain-string templates with slots: {target}, {room}, {tick}, {self}. Minimum set: 3 accusation lines, 2 alibi lines, 2 defence lines, 2 grudge lines, 2 chat-reveal lines, per archetype where the tone differs. ~40 lines total. Templates chosen by archetype and suspicion band (low/mid/high).

5.5 Art & audio (placeholders first, real assets in M6)

- 16×16 or 32×32 top-down tileset (Kenney or similar CC0), one tilemap built in Tiled.
- 7 portraits (player + 6 NPCs), 7 walk sprites. Placeholder: coloured rectangles with initials.
- SFX: footstep, sneak, discovery sting, vote drumroll, banish, murder. Music: one looping ambient track.

6. Milestones

Dates assume a 2 Nov start. Each milestone ends with a commit tagged *mX* and a playable build. If a milestone slips by more than 2 days, cut from the **Stretch** list first, then from M6.

M0 — Tooling (2 Nov, max 1 day)

- Create repo *traitors* (public). Vite + TypeScript + Phaser 3 template. Vitest for *src/sim*.
- ESLint + Prettier. GitHub Actions: on push, run tests and build; on tag, publish *dist/* as a release zip.
- README with the one-paragraph pitch and the AI-use disclosure the jam asks for.

Done when:

- npm run dev shows a Phaser canvas with a moving rectangle.
- npm test runs one passing sim test. CI green.

M1 — Simulation core (3–6 Nov)

- Implement *types.ts, rng.ts, world.ts, npc.ts, sightings.ts, evidence.ts, game.ts* with *applyCommand*.
- Load content JSON. Implement Day phase commands: move, pickUp, plant, endDay. Implement discovery and sightings.
- Implement Round Table vote resolution (4.4) and Night (4.5). Dialogue can be raw strings for now.
- Write a headless *scripts/sim-cli.ts* that plays a run from a seed and prints the log.

Done when:

- Test: planting the Cellar key in the library then having Marcus walk through raises Marcus.suspicion[theo] by 25.
- Test: player seen (non-sneaking) by Marcus in the cellar, then murder in the cellar → Marcus.suspicion[player] += 30.
- Test: a full 6-day run from seed 42 completes with an outcome and no exceptions.
- Test: Loyalist votes with ally; Sheep follows majority; ties go against player.

M2 — Mansion scene (7–10 Nov)

- Tiled map with the 9 rooms. Player sprite with WASD/arrows; Shift halves speed and sets *sneaking*.
- NPC sprites that teleport (fade out/in) to *schedule[tick]* rooms. Sight cones (60°, 4 tiles) drawn as translucent fans; cone hit calls *sim.recordSighting*.
- Room entry triggers; HUD shows day, tick, actions left, current room.
- Interact key (E) opens a DOM context menu: pick up / plant / talk-to-NPC.

Done when:

- Walk from hall to cellar; HUD updates; cone intersection logs a sighting in the sim log.
- Holding Shift visibly slows the player and the sim marks moves as sneaking.

M3 — Dialogue, rumours, Round Table UI (11–16 Nov)

- DOM dialogue panel with the 4 verbs; Shift-held reveals the lie variants and logs them.
- Rumour propagation (4.3). Templates (5.4) wired through *speak.ts*.
- Round Table screen: portraits in a circle, NPC speeches typed out, player statement menu, vote tally animation, banish.
- Trust web (SVG lines between portraits; thickness = |trust|, colour = sign) shown before and after the vote.
- Night screen: pick victim + room. Morning announcement of the body.

Done when:

- A full day can be played end-to-end in the browser with placeholder art.
- Seeding a lie that the Detective later contradicts produces a visible accusation at the table.

M4 — Playable loop & win/lose (17–19 Nov)

- Title screen, seed display, restart. End screen with a replay of the vote history.
- Save/resume via localStorage (state is serialisable JSON).
- First external playtest with 3 people. Record: time to first vote, whether they understood why they were voted out.

Done when:

- 3 playtesters complete a run without the developer explaining anything beyond the controls.

M5 — Balance & feel (20–23 Nov)

- Tune *balance.ts* using the sim CLI: run 200 seeded games with a scripted ‘naive’ player and a ‘careful’ player; target win rates ~25% and ~55%.
- Add tells: NPCs glance at the player when suspicion is high; rumour whispers float as text in rooms.
- Tutorial day 0: scripted first day that forces one plant and one sneak.

Done when:

- Win-rate targets hit. Playtesters can name at least one reason they were suspected.

M6 — Art, audio, polish (24–27 Nov)

- Replace placeholders: tileset, sprites, portraits, title logo with the ‘AI’ letters highlighted.
- SFX and music. Screen shake on murder reveal, drumroll on vote.
- itch.io page: cover image, 3 screenshots, GIF of a sneak, description, controls, AI-use disclosure.

Done when:

- Build runs on itch.io in Chrome, Firefox and Safari at 960×540 and fullscreen.

M7 — Ship (28–29 Nov; hard stop 1 Dec 21:37 UK)

- Tag v1.0. Upload HTML5 zip to itch.io, submit to the jam. Push final code. Post in the jam community thread.

Done when:

- Submission confirmed on itch.io *and* code on GitHub before the deadline.

7. Stretch list (only after M6) and cut order

Stretch, in priority order:

- Mission room: a 45-second timing mini-game once per day; success grants a Shield (immune to banishment once) that NPCs then question (+10 suspicion from Paranoid).
- Second traitor NPC the player can recruit on night 2 (shares blame, may betray).
- Faithful mode.
- Accessibility: remappable keys, toggle-sneak option instead of hold.

If behind schedule, cut in this order:

- 1. Music (keep SFX). 2. Save/resume. 3. Rosa (run with 5 NPCs). 4. Rumour propagation between NPCs (player-seeded rumours only). 5. Tutorial day (replace with a text card). Never cut: Shift-sneak, Shift-lie, evidence provenance, the trust web.

8. Prompt templates for AI coding assistants

Paste Sections 1–4 of this document, then one of these:

```
[M1] Implement src/sim per the data model and rules. Pure TypeScript, no Phaser
imports. Export applyCommand(state, command) as a pure function and
createGame(seed, content) to build the initial state from the JSON files. Write
Vitest tests for every "Done when" item in M1. Do not implement UI.

[M2] Using Phaser 3 + Vite + TypeScript, build scenes/Mansion.ts: load the Tiled
map in public/assets/mansion.json, add a player with WASD movement where holding
Shift halves speed and sets sneaking=true, render NPC sprites at
sim.state.npcs[id].schedule[tick], draw 60-degree sight cones of 4 tiles, and
call sim.applyCommand({type:'move',...}) on room change. Keep all game rules in
src/sim; the scene only renders state and sends commands.

[M3] Build the DOM overlay in src/ui. dialogue.ts shows the 4 verbs for the
NPC in the player's room; when the Shift key is held, show the lie variant of
seedDoubt and set isLie=true. roundTable.ts renders portraits in a circle,
types out speeches from sim.roundTableSpeeches(), shows the player statement
menu, animates the tally and the banishment. trustWeb.ts draws an SVG web from
npc.trusts. No game rules in the UI.
```

Review checklist to run after each AI-generated milestone: no Phaser import under *src/sim*; every random call uses *rng.ts*; *applyCommand* never mutates its input; all tunable numbers live in *balance.ts*; tests pass; a seeded CLI run still completes.